

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **Thamaraiselvi S (192511042)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

The drone must deliver medicines quickly in a flood-affected area where roads may be blocked and weather conditions can change the travel cost. It has a straight-line distance heuristic and partial real-time information.

i. Greedy Best-First Search

Greedy Best-First Search selects the next node based mainly on the heuristic value $h(n)$. In this case, the heuristic can be the straight-line distance to the destination.

ii. A* Search

A* considers both the actual cost and estimated remaining cost.

Formula:

$$f(n) = g(n) + h(n)$$

Where:

- * $g(n)$ = actual cost from the starting point.

- * $h(n)$ = estimated cost to the destination.

- * $f(n)$ = total estimated cost.

Therefore, A* can consider distance, weather, terrain and delay while selecting a route.

iii. Effect of Heuristic Accuracy

A good heuristic helps both algorithms search faster and choose better routes. Greedy Search depends heavily on the heuristic, so an inaccurate heuristic can lead to a poor route. A* is more balanced because it also considers the actual cost.

iv. Dynamic Changes

When a road becomes blocked or weather conditions change, the previous route may no longer be suitable. The algorithm must update the path information and search for another route. Frequent changes can increase search time.

v. Recommendation

A* is recommended* because it balances actual travel cost and estimated distance. It provides a better combination of speed, optimality and safety compared with Greedy Best-First Search.

2. Smart City Traffic Signal Optimization

The objective is to optimize traffic signals to reduce waiting time, fuel consumption and traffic congestion. The search space is very large and traffic conditions continuously change.

i. Hill Climbing

Hill Climbing starts with an initial signal arrangement and changes it to a better neighboring arrangement. It continues until no better neighboring solution is found. It is simple and fast, but it can get stuck at a local optimum and may not find the best overall solution.

ii. Local Maxima, Plateaus and Ridges

- * Local maximum: A signal arrangement that is better than nearby solutions but not the best overall.

- * Plateau: Many nearby solutions have almost the same performance.

- * Ridge: Improvement may require several changes together rather than one simple change.

These conditions can cause Hill Climbing to stop too early.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing can temporarily accept a worse solution. This allows it to escape local optima and explore other solutions. Genetic Algorithm maintains several possible solutions and uses selection, crossover and mutation to generate better solutions.

iv. Recommendation

Genetic Algorithm is a suitable choice for a large smart-city system because it can explore many solutions and adapt to changing traffic patterns. Simulated Annealing can also be useful when avoiding local optima is the main concern.

3. Autonomous Mars Rover – Online Search

The rover operates in unknown terrain, has limited sensing capability, and continuously updates its knowledge while exploring.

i. Need for Online Search

An offline search requires knowledge of the environment before searching. The Mars rover does not have a complete map, so it must discover information while moving. Therefore, an online search agent is more suitable because it makes decisions while interacting with the environment.

ii. Challenges

The rover faces:

- * Partial observability.
- * Unknown rocks and craters.
- * Limited sensor information.
- * Incomplete maps.
- * New information during exploration.

These conditions require the rover to continuously change its decisions.

iii. Internal Model

The rover uses sensors to observe the surrounding terrain. It records obstacles, safe areas and possible routes. As it explores, it updates its internal map and uses the new information to make future decisions.

iv. Comparison

Uninformed search does not use additional heuristic information. Informed search uses heuristic information to guide the search. Online search learns about the environment while moving and searching. For an unknown Mars environment, online search is especially useful.

v. Recommendation

Online search is the most suitable approach because the rover needs adaptability, safety and the ability to handle uncertainty. It can update its knowledge whenever new terrain information is discovered,

4. University Exam Timetable – Constraint Satisfaction Problem

The university timetable has many restrictions such as no overlapping exams, limited halls, faculty availability and special student requirements. Therefore, it can be formulated as a Constraint Satisfaction Problem (CSP).

i. Variables, Domains and Constraints

Variables : Each course or examination is a variable.

Domains : Available exam time slots and examination halls.

Constraints : Some subjects must be scheduled before others.

ii. Backtracking Search

Backtracking assigns a time and hall to each exam one by one. If a conflict occurs, the algorithm goes back to the previous assignment and tries another available value. This continues until a timetable satisfying all constraints is obtained.

iii. Constraint Propagation / Forward Checking

Forward checking removes values that are no longer possible from the domains of unassigned variables.

For example, if two exams have common students, the same time slot can be removed from one exam after assigning it to the other. This detects conflicts early and reduces unnecessary searching.

iv. Recommendation

Backtracking with constraint propagation is suitable for this problem. It can reduce the search space and handle many constraints more efficiently as the number of courses and students increases.

5. Strategic Game AI – Minimax and Alpha-Beta Pruning

The gaming AI must compete with human players, evaluate many possible moves and make decisions within strict time limits. The large game tree makes complete searching expensive.

i. Minimax Algorithm

Minimax is used for two-player competitive games.

* MAX player: The AI tries to maximize its score.

* MIN player: The opponent tries to minimize the AI's score.

ii. Computational Challenges

Game trees can become extremely large because every state may have many possible moves. Searching every possible state requires a large amount of time and memory. Therefore, complete Minimax search may not be suitable for real-time games.

iii. Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by removing branches that cannot affect the final decision.

* Alpha: Best value that MAX can guarantee.

* Beta: Best value that MIN can guarantee.

iv. Evaluation Function

Since the complete game cannot always be searched, an evaluation function can estimate the quality of a game state.

It can consider:

- * Resource advantage.
- * Territory or position.
- * Military strength
- * Defensive safety.

A suitable combination of these factors helps the AI choose better moves.

v. Recommendation

For real-time performance, the best approach is depth-limited Minimax with Alpha-Beta pruning and an evaluation function. Depth limitation controls the search time, Alpha-Beta reduces unnecessary branches, and the evaluation function estimates the best move when the search cannot reach the final game state.